

Propuesta de Proyecto Final

Detector de Fraude en Transacciones Bancarias mediante Análisis de Ciclos en Grafos

Curso: Algoritmos y Estructuras de Datos

Equipo: Ma Fabiana Sánchez, Corina Vezga y Victoria Pereira

Fecha: 9 de julio 2026

1. Objetivo del proyecto

Desarrollar un programa en C que modele un conjunto de cuentas bancarias y sus transacciones como un grafo dirigido, y que detecte automáticamente ciclos de transacciones (A envía a B, B envía a C, C envía de vuelta a A) como indicador de posibles patrones de fraude o lavado de dinero, priorizando los casos más relevantes para su revisión.

2. Justificación

Nos basamos en el artículo "The detection of professional fraud in automobile insurance using social network analysis" (Bodaghi & Teimourpour), que modela accidentes automovilísticos como un grafo de conductores y utiliza DFS/BFS para detectar ciclos en la red como evidencia de fraude organizado, en lugar de usar comunidades densas. Su argumento central es que en fraude coordinado los mismos actores "cierran el círculo" entre ellos, y que los ciclos son mucho más específicos y baratos de calcular que otros métodos de detección de comunidades.

Adaptamos esa misma idea al dominio bancario: en lugar de conductores/choques, usamos cuentas/transacciones. Un ciclo de transacciones (dinero que vuelve a su origen tras pasar por varias cuentas) es un patrón clásico y bien documentado de lavado de dinero, por lo que el enfoque del paper es directamente aplicable.

3. Alcance

Incluye:

- Lectura de transacciones desde un archivo de texto (origen, destino, monto).
- Construcción de un grafo dirigido y ponderado mediante lista de adyacencia (TAD).
- Detección de ciclos mediante DFS (algoritmo visto en clase).
- Filtrado de ciclos triviales (longitud menor a 3, ajustado a nuestro caso de estudio).
- Cálculo del monto total involucrado en cada ciclo.
- Ordenamiento de los ciclos por monto total usando Quick Sort, para priorizar cuáles revisar primero.
- Interfaz de consola que muestra el grafo cargado y los ciclos sospechosos.

No incluye (fuera de alcance):

- Conexión a bases de datos reales o APIs bancarias.
- Machine learning o modelos probabilísticos de fraude.
- Interfaz gráfica (opcionalmente se podría agregar sin afectar la lógica central en C puro).

4. Resultados esperados

- Un programa en C que compila sin errores ni warnings (-Wall -Wextra) y sin fugas de memoria.
- Dado un archivo de transacciones de prueba, el programa debe listar correctamente todos los ciclos de longitud ≥ 3 , ordenados de mayor a menor monto total.

- Un informe técnico que documente la complejidad Big-O de DFS ($O(V+E)$) y de Quick Sort ($O(n \log n)$ promedio, $O(n^2)$ peor caso), y justifique la elección de la lista de adyacencia sobre la matriz de adyacencia.
- Una demostración funcional de 15 minutos mostrando el flujo completo: carga de datos, construcción del grafo, detección y priorización de ciclos.

5. Algoritmos y TAD utilizados

Elemento	Elección	Justificación breve
TAD principal	Grafo dirigido (lista de adyacencia)	Las transacciones son dispersas (pocas conexiones por cuenta comparado con el total), por lo que la lista de adyacencia es más eficiente en espacio que la matriz de adyacencia
Algoritmo core #1	DFS para detección de ciclos	Mismo enfoque validado en el paper de referencia; complejidad $O(V+E)$
Algoritmo core #2	Quick Sort	Para priorizar ciclos por monto total; complejidad $O(n \log n)$ promedio

6. Cronograma interno del equipo

Fecha	Tarea
9 jul	Entrega de esta propuesta
10–12 jul	Implementación del TAD grafo y carga de datos
13–15 jul	Implementación de detección de ciclos (DFS) y pruebas
16 jul	Implementación de Quick Sort y priorización
17 jul	Pruebas finales, documentación técnica, README
18 jul (12:00 pm)	Entrega final en GitHub
19–20 jul	Preparación y defensa oral